

Notebook_ca4

November 28, 2025

1 IND320 Course Project, Part 4

1.1 Code access and direct links

- The project is deployed here: [ind320-henrikengdal-project](#)
- The code is accessible at the repository: [henrikengdal/ind320-henrikengdal-project](#)

1.2 AI Usage

AI plays a multifaceted role throughout this project, primarily serving as an assistant and analytical tool. The project leverages AI in several areas:

Development and Code Generation: AI assists in writing and optimizing code for the application.

Data Analysis and Insights: AI helps analyze data patterns and identifying trends. It assists in generating meaningful statistical summaries and suggesting appropriate visualization techniques for the given data.

Documentation and Communication: AI supports the creation of clear documentation, such as code comments, and user interface text. It helps structure the project documentation and ensures technical concepts are communicated effectively.

Problem-Solving and Debugging: Throughout the development process, AI serves as a coding companion, helping troubleshoot issues, optimize data processing workflows, and suggesting best practices.

1.3 Retrieving production data from Elhub API

1.3.1 Imports

```
[3]: import requests
import json
from datetime import datetime, timedelta
```

1.3.2 Defining function to fetch data of given month

```
[4]: # Function to fetch data for a specific month
def fetch_month_data(year, month, dataset):
    start_date = datetime(year, month, 1)
```

```

# Calculate last day of month
if month == 12:
    end_date = datetime(year + 1, 1, 1) - timedelta(days=1)
else:
    end_date = datetime(year, month + 1, 1) - timedelta(days=1)

# Format dates for API (URL encoded)
start_str = start_date.strftime('%Y-%m-%dT00:00:00+02:00').replace(':', '␣', 1)
↪ '%3A').replace('+', '%2B')
end_str = end_date.strftime('%Y-%m-%dT23:59:59+02:00').replace(':', '%3A').
↪ replace('+', '%2B')

# Build URL
url = f"https://api.elhub.no/energy-data/v0/price-areas?
↪ dataset={dataset}&startDate={start_str}&endDate={end_str}"

try:
    response = requests.get(url)
    response.raise_for_status()
    return response.json()
except Exception as e:
    print(f"Error fetching {start_date.strftime('%B %Y')}: {e}")
    return None

```

1.3.3 Defining function to fetch data of given year

```

[5]: def fetch_year_data(year, dataset):
    for month in range(1, 13):
        month_data = fetch_month_data(year, month, dataset)

    if month_data and 'data' in month_data:
        if combined_response is None:
            # First month - use as template
            combined_response = month_data
        else:
            # Add data from subsequent months
            # Combine the productionPerGroupMbaHour arrays for each price area
            for new_area in month_data['data']:
                # Find matching area in combined data
                area_found = False
                for existing_area in combined_response['data']:
                    if existing_area['attributes']['name'] ==␣
↪ new_area['attributes']['name']:
                        # Append production data
                        ␣
↪ existing_area['attributes']['productionPerGroupMbaHour'].extend(
                            new_area['attributes']['productionPerGroupMbaHour']

```

```

        )
        area_found = True
        break

    # If area not found, add it
    if not area_found:
        combined_response['data'].append(new_area)

    print(f"{datetime(year, month, 1).strftime('%B')} - Fetched_
↪successfully")
    else:
        print(f"{datetime(year, month, 1).strftime('%B')} - No data received")

```

1.3.4 Fetching and saving data for period 2022-2024

```

[6]: # Fetch and combine data for 2021-2024, then save to a single file
def fetch_and_combine_years(years, dataset, attribute, write_path):
    combined_response = None
    for year in years:
        for month in range(1, 13):
            month_data = fetch_month_data(year, month, dataset)
            if month_data and 'data' in month_data:
                if combined_response is None:
                    combined_response = month_data
                else:
                    for new_area in month_data['data']:
                        area_found = False
                        for existing_area in combined_response['data']:
                            if existing_area['attributes']['name'] ==_
↪new_area['attributes']['name']:
                                existing_area['attributes'][attribute].extend(
                                    new_area['attributes'][attribute]
                                )
                                area_found = True
                                break
                        if not area_found:
                            combined_response['data'].append(new_area)
                            print(f"{datetime(year, month, 1).strftime('%B_
↪%Y')} - Fetched successfully")
                        else:
                            print(f"{datetime(year, month, 1).strftime('%B %Y')} - No data_
↪received")

    if combined_response:
        file_path = write_path
        with open(file_path, 'w') as f:
            json.dump(combined_response, f, indent=4)

```

```

        total_records = sum(
            len(area['attributes'][attribute])
            for area in combined_response['data']
        )
        print(f"\nSuccessfully saved all data for years {years}!")
        print(f"Total records: {total_records:,}")
        print(f"File saved to: {file_path}")
    else:
        print(f" No data was fetched for years {years}. Please check the API
↳or your internet connection.")

# Fetch and save data into one file
Production_dataset = "PRODUCTION_PER_GROUP_MBA_HOUR"
Production_write_path = './assets/ELHUB_Production_data.json'
Production_attribute = "productionPerGroupMbaHour"
fetch_and_combine_years([2021, 2022, 2023, 2024], Production_dataset,
↳Production_attribute, Production_write_path)

Consumption_dataset = "CONSUMPTION_PER_GROUP_MBA_HOUR"
Consumption_write_path = './assets/ELHUB_Consumption_data.json'
Consumption_attribute = "consumptionPerGroupMbaHour"
fetch_and_combine_years([2021, 2022, 2023, 2024], Consumption_dataset,
↳Consumption_attribute, Consumption_write_path)

```

Successfully saved all data for years [2021, 2022, 2023, 2024]!
 Total records: 872,953
 File saved to: ./assets/ELHUB_Production_data.json

Successfully saved all data for years [2021, 2022, 2023, 2024]!
 Total records: 876,600
 File saved to: ./assets/ELHUB_Consumption_data.json

1.4 Converting the data to a Dataframe

```

[7]: # Convert the JSON data into a pandas DataFrame
import pandas as pd

# Load the JSON file
with open('./assets/ELHUB_Production_data.json', 'r') as f:
    production_data = json.load(f)

with open('./assets/ELHUB_Consumption_data.json', 'r') as f:
    consumption_data = json.load(f)

def extract_records(data, attribute):

```

```

records = []
for item in data['data']:
    for entry in item['attributes'][attribute]:
        records.append(entry)
return records

production_records = extract_records(production_data, Production_attribute)
consumption_records = extract_records(consumption_data, Consumption_attribute)

# Create DataFrame
df_production = pd.DataFrame(production_records)
df_consumption = pd.DataFrame(consumption_records)

# Convert time columns to datetime
df_production['endTime'] = pd.to_datetime(df_production['endTime'], utc=True).
    ↪dt.tz_localize(None)
df_production['lastUpdatedTime'] = pd.
    ↪to_datetime(df_production['lastUpdatedTime'], utc=True).dt.tz_localize(None)
df_production['startTime'] = pd.to_datetime(df_production['startTime'],
    ↪utc=True).dt.tz_localize(None)

df_consumption['endTime'] = pd.to_datetime(df_consumption['endTime'], utc=True).
    ↪dt.tz_localize(None)
df_consumption['lastUpdatedTime'] = pd.
    ↪to_datetime(df_consumption['lastUpdatedTime'], utc=True).dt.tz_localize(None)
df_consumption['startTime'] = pd.to_datetime(df_consumption['startTime'],
    ↪utc=True).dt.tz_localize(None)

print(df_production.head())
print(df_consumption.head())

```

	endTime	lastUpdatedTime	priceArea	productionGroup	\
0	2021-01-01 00:00:00	2024-12-20 09:35:40	N01	hydro	
1	2021-01-01 01:00:00	2024-12-20 09:35:40	N01	hydro	
2	2021-01-01 02:00:00	2024-12-20 09:35:40	N01	hydro	
3	2021-01-01 03:00:00	2024-12-20 09:35:40	N01	hydro	
4	2021-01-01 04:00:00	2024-12-20 09:35:40	N01	hydro	

	quantityKwh	startTime
0	2507716.8	2020-12-31 23:00:00
1	2494728.0	2021-01-01 00:00:00
2	2486777.5	2021-01-01 01:00:00
3	2461176.0	2021-01-01 02:00:00
4	2466969.2	2021-01-01 03:00:00

	consumptionGroup	endTime	lastUpdatedTime	\
0	cabin	2021-01-01 00:00:00	2024-12-20 09:35:40	
1	cabin	2021-01-01 01:00:00	2024-12-20 09:35:40	

```

2          cabin 2021-01-01 02:00:00 2024-12-20 09:35:40
3          cabin 2021-01-01 03:00:00 2024-12-20 09:35:40
4          cabin 2021-01-01 04:00:00 2024-12-20 09:35:40

```

```

      meteringPointCount priceArea  quantityKwh      startTime
0          100607         NO1    177071.56 2020-12-31 23:00:00
1          100607         NO1    171335.12 2021-01-01 00:00:00
2          100607         NO1    164912.02 2021-01-01 01:00:00
3          100607         NO1    160265.77 2021-01-01 02:00:00
4          100607         NO1    159828.69 2021-01-01 03:00:00

```

1.5 Setting up Cassandra and Spark Connection

```

[8]: from cassandra.cluster import Cluster
     from cassandra.query import BatchStatement

def pandas_to_cassandra_type(dtype):
    if pd.api.types.is_integer_dtype(dtype):
        return 'int'
    elif pd.api.types.is_float_dtype(dtype):
        return 'float'
    elif pd.api.types.is_datetime64_any_dtype(dtype):
        return 'timestamp'
    else:
        return 'text'

def insert_into_cassandra(df, keyspace, table_name):
    df['row_id'] = range(1, len(df) + 1)
    columns = df.columns
    primary_key = columns[-1]

    cluster = Cluster(['localhost'], port=9042)
    session = cluster.connect()
    session.set_keyspace(keyspace)

    columns_cql = ', '.join([
        f'{col} {pandas_to_cassandra_type(df[col].dtype)}' for col in columns
    ])

    session.execute(f"DROP TABLE IF EXISTS {keyspace}.{table_name};")
    create_table_cql = f"""
CREATE TABLE IF NOT EXISTS {table_name} (
    {columns_cql},
    PRIMARY KEY ({primary_key})
)
"""
    session.execute(create_table_cql)

```

```

print(f"Table {table_name} created successfully.")

columns = list(columns)
placeholders = ', '.join(['?'] * len(columns))
columns_str = ', '.join(columns)

insert_cql = f"INSERT INTO {table_name} ({columns_str}) VALUES_
↳({placeholders})"

BATCH_SIZE = 100

prepared = session.prepare(insert_cql)
batch = BatchStatement()
for index, row in df.iterrows():
    values = [row[col] for col in columns]
    batch.add(prepared, values)

    if (index + 1) % BATCH_SIZE == 0:
        session.execute(batch)
        batch.clear()
if batch:
    session.execute(batch)

print(f"Data inserted into {table_name} successfully.")

insert_into_cassandra(df_production, 'ind320_project', 'production')
insert_into_cassandra(df_consumption, 'ind320_project', 'consumption')

```

Table production created successfully.
Data inserted into production successfully.
Table consumption created successfully.
Data inserted into consumption successfully.

```

[9]: import os
os.environ["JAVA_HOME"] = "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/
↳Contents/Home"
os.environ["PYSPARK_PYTHON"] = "python"
print(f"JAVA_HOME set to: {os.environ['JAVA_HOME']}")

# Verify Java is accessible
import subprocess
try:
    result = subprocess.run([f"{os.environ['JAVA_HOME']}/bin/java", "-version"],
                            capture_output=True, text=True, timeout=5)
    print(f"Java version: {result.stderr.split(chr(10))[0]}")
except Exception as e:

```

```
print(f"Error checking Java: {e}")
```

JAVA_HOME set to: /opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home

Java version: openjdk version "17.0.17" 2025-10-21

```
[10]: from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('SparkCassandraApp').\
    config('spark.jars.packages', 'com.datastax.spark:\
    ↪spark-cassandra-connector_2.12:3.5.1,org.mongodb.spark:\
    ↪mongo-spark-connector_2.12:10.2.0').\
    config('spark.cassandra.connection.host', 'localhost').\
    config('spark.sql.extensions', 'com.datastax.spark.connector.\
    ↪CassandraSparkExtensions').\
    config('spark.sql.catalog.mycatalog', 'com.datastax.spark.connector.\
    ↪datasource.CassandraCatalog').\
    config('spark.cassandra.connection.port', '9042').getOrCreate()

print("Spark session connected to Cassandra.")
```

Spark session connected to Cassandra.

1.6 Extracting Columns with Spark

```
[11]: from pyspark.sql.functions import sum as spark_sum, col, avg as spark_avg, \
    ↪count, max as spark_max

# Convert pandas DataFrame to Spark DataFrame
spark_production_df = spark.createDataFrame(df_production)
spark_consumption_df = spark.createDataFrame(df_consumption)

def extract_columns_with_spark(spark_df, column):

    spark_df.printSchema()
    spark_df.show(5)

    selected_columns = spark_df.select('priceArea', column, 'quantityKwh', \
    ↪'startTime', 'endTime', 'lastUpdatedTime')
    selected_columns.show(5)

# Aggregation 1: Total production by group and price area
production_summary = spark_df.groupBy(column, 'priceArea') \
    .agg(spark_sum('quantityKwh').alias('total_production')) \
    .orderBy(col('total_production').desc())

production_summary.show()
```

```

# Aggregation 2: Additional statistics per production group
production_stats = spark_df.groupBy(column) \
    .agg(
        spark_sum('quantityKwh').alias('total_kwh'),
        spark_avg('quantityKwh').alias('avg_kwh'),
        spark_max('quantityKwh').alias('max_kwh'),
        count('*').alias('record_count')
    ) \
    .orderBy(col('total_kwh').desc())

print("\nProduction Statistics by Group:")
production_stats.show()

# Convert Spark results to pandas for visualization
production_summary_pd = production_summary.toPandas()
production_stats_pd = production_stats.toPandas()

print("\n Spark aggregations completed")
print(f"Summary records: {len(production_summary_pd)}")
print(f"Stats records: {len(production_stats_pd)}")
print("\nSample summary data:")
print(production_summary_pd.head(10))

extract_columns_with_spark(spark_production_df, 'productionGroup')
extract_columns_with_spark(spark_consumption_df, 'consumptionGroup')

```

```

root
|-- endTime: timestamp (nullable = true)
|-- lastUpdatedTime: timestamp (nullable = true)
|-- priceArea: string (nullable = true)
|-- productionGroup: string (nullable = true)
|-- quantityKwh: double (nullable = true)
|-- startTime: timestamp (nullable = true)
|-- row_id: long (nullable = true)

```

25/11/27 09:19:41 WARN TaskSetManager: Stage 0 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

25/11/27 09:19:46 WARN PythonRunner: Detected deadlock while completing task 0.0 in stage 0 (TID 0): Attempting to kill Python Worker

```

+-----+-----+-----+-----+-----+
|          endTime|    lastUpdatedTime|priceArea|productionGroup|quantityKwh|
startTime|row_id|
+-----+-----+-----+-----+-----+
|2021-01-01 00:00:00|2024-12-20 09:35:40|      N01|          hydro|

```

```

2507716.8|2020-12-31 23:00:00|      1|
|2021-01-01 01:00:00|2024-12-20 09:35:40|      N01|      hydro|
2494728.0|2021-01-01 00:00:00|      2|
|2021-01-01 02:00:00|2024-12-20 09:35:40|      N01|      hydro|
2486777.5|2021-01-01 01:00:00|      3|
|2021-01-01 03:00:00|2024-12-20 09:35:40|      N01|      hydro|
2461176.0|2021-01-01 02:00:00|      4|
|2021-01-01 04:00:00|2024-12-20 09:35:40|      N01|      hydro|
2466969.2|2021-01-01 03:00:00|      5|
+-----+-----+-----+-----+-----+
-----+
only showing top 5 rows

```

25/11/27 09:19:46 WARN TaskSetManager: Stage 1 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

25/11/27 09:19:50 WARN PythonRunner: Detected deadlock while completing task 0.0 in stage 1 (TID 1): Attempting to kill Python Worker

```

+-----+-----+-----+-----+-----+
-----+
|priceArea|productionGroup|quantityKwh|      startTime|      endTime|
lastUpdatedTime|
+-----+-----+-----+-----+-----+
-----+
|      N01|      hydro| 2507716.8|2020-12-31 23:00:00|2021-01-01
00:00:00|2024-12-20 09:35:40|
|      N01|      hydro| 2494728.0|2021-01-01 00:00:00|2021-01-01
01:00:00|2024-12-20 09:35:40|
|      N01|      hydro| 2486777.5|2021-01-01 01:00:00|2021-01-01
02:00:00|2024-12-20 09:35:40|
|      N01|      hydro| 2461176.0|2021-01-01 02:00:00|2021-01-01
03:00:00|2024-12-20 09:35:40|
|      N01|      hydro| 2466969.2|2021-01-01 03:00:00|2021-01-01
04:00:00|2024-12-20 09:35:40|
+-----+-----+-----+-----+-----+
-----+
only showing top 5 rows

```

25/11/27 09:19:51 WARN TaskSetManager: Stage 2 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

25/11/27 09:19:52 WARN TaskSetManager: Stage 5 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

```

+-----+-----+-----+
|productionGroup|priceArea|      total_production|
+-----+-----+-----+
|      hydro|      N02|1.872465471438301...|
|      hydro|      N05|1.202250220871899...|

```

	hydro	N04	9.213612001844986E10
	hydro	N03	7.860196346273007E10
	hydro	N01	7.331910698147005E10
	wind	N03	2.308311771201900...
	wind	N02	1.678158483858005...
	wind	N04	1.072251232590594...
	thermal	N04	5.015641029794993E9
	wind	N01	3.1986961571950045E9
	thermal	N05	1.4955147956690006E9
	thermal	N01	9.895174540730076E8
	thermal	N02	6.786122238399982E8
	thermal	N03	3.934946759600018E8
	solar	N01	2.3861903965399963E8
	solar	N02	2.0297909434900117E8
	solar	N05	3.5476559022000164E7
	solar	N03	3.201269785199999E7
	other	N04	1.7896367182000663E7
	other	N02	4392107.913000019

+-----+-----+-----+-----+

only showing top 20 rows

Production Statistics by Group:

+-----+-----+-----+-----+			
-+			
productionGroup	total_kwh	avg_kwh	
max_kwh	record_count		
+-----+-----+-----+-----+			
-+			
	hydro	5.515287596936703E11	3145840.518444389 1.0054233E7
175320			
	wind	5.378591602344199E10	313304.45686533116 1916776.5
171673			
	thermal	8.572780179337E9	48897.902003975585 264177.0
175320			
	solar	5.107173442639999E8	2913.058089573351 138293.64
175320			
	other	2.744070201000077E7	156.5178074948709 9667.712
175320			
+-----+-----+-----+-----+			
-+			

25/11/27 09:19:53 WARN TaskSetManager: Stage 8 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

25/11/27 09:19:55 WARN TaskSetManager: Stage 16 contains a task of very large

size (6056 KiB). The maximum recommended task size is 1000 KiB.

Spark aggregations completed
Summary records: 25
Stats records: 5

Sample summary data:

	productionGroup	priceArea	total_production
0	hydro	N02	1.872465e+11
1	hydro	N05	1.202250e+11
2	hydro	N04	9.213612e+10
3	hydro	N03	7.860196e+10
4	hydro	N01	7.331911e+10
5	wind	N03	2.308312e+10
6	wind	N02	1.678158e+10
7	wind	N04	1.072251e+10
8	thermal	N04	5.015641e+09
9	wind	N01	3.198696e+09

root

```
|-- consumptionGroup: string (nullable = true)
|-- endTime: timestamp (nullable = true)
|-- lastUpdatedTime: timestamp (nullable = true)
|-- meteringPointCount: long (nullable = true)
|-- priceArea: string (nullable = true)
|-- quantityKwh: double (nullable = true)
|-- startTime: timestamp (nullable = true)
|-- row_id: long (nullable = true)
```

25/11/27 09:19:56 WARN TaskSetManager: Stage 24 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

25/11/27 09:20:00 WARN PythonRunner: Detected deadlock while completing task 0.0 in stage 24 (TID 42): Attempting to kill Python Worker

25/11/27 09:20:00 WARN TaskSetManager: Stage 25 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

```
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
|consumptionGroup|          endTime|
lastUpdatedTime|meteringPointCount|priceArea|quantityKwh|
startTime|row_id|
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
|          cabin|2021-01-01 00:00:00|2024-12-20 09:35:40|          100607|
N01| 177071.56|2020-12-31 23:00:00|          1|
|          cabin|2021-01-01 01:00:00|2024-12-20 09:35:40|          100607|
N01| 171335.12|2021-01-01 00:00:00|          2|
|          cabin|2021-01-01 02:00:00|2024-12-20 09:35:40|          100607|
```

```

N01| 164912.02|2021-01-01 01:00:00| 3|
| cabin|2021-01-01 03:00:00|2024-12-20 09:35:40| 100607|
N01| 160265.77|2021-01-01 02:00:00| 4|
| cabin|2021-01-01 04:00:00|2024-12-20 09:35:40| 100607|
N01| 159828.69|2021-01-01 03:00:00| 5|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 5 rows

```

25/11/27 09:20:04 WARN PythonRunner: Detected deadlock while completing task 0.0 in stage 25 (TID 43): Attempting to kill Python Worker

25/11/27 09:20:04 WARN TaskSetManager: Stage 26 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

```

+-----+-----+-----+-----+-----+
+-----+
|priceArea|consumptionGroup|quantityKwh|          startTime|          endTime|
lastUpdatedTime|
+-----+-----+-----+-----+-----+
+-----+
|      N01|      cabin| 177071.56|2020-12-31 23:00:00|2021-01-01
00:00:00|2024-12-20 09:35:40|
|      N01|      cabin| 171335.12|2021-01-01 00:00:00|2021-01-01
01:00:00|2024-12-20 09:35:40|
|      N01|      cabin| 164912.02|2021-01-01 01:00:00|2021-01-01
02:00:00|2024-12-20 09:35:40|
|      N01|      cabin| 160265.77|2021-01-01 02:00:00|2021-01-01
03:00:00|2024-12-20 09:35:40|
|      N01|      cabin| 159828.69|2021-01-01 03:00:00|2021-01-01
04:00:00|2024-12-20 09:35:40|
+-----+-----+-----+-----+-----+
+-----+
only showing top 5 rows

```

25/11/27 09:20:05 WARN TaskSetManager: Stage 29 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

```

+-----+-----+-----+
|consumptionGroup|priceArea|    total_production|
+-----+-----+-----+
|      secondary|      N02| 7.67067486067005E10|
|      secondary|      N03|6.213563897620004E10|
|      household|      N01|6.069843108294979E10|
|      tertiary|      N01|4.097971911068008...|
|      secondary|      N05|3.758355819151035E10|
|      secondary|      N04|3.724764449405984E10|
|      household|      N02|3.460145224478002E10|
|      household|      N03|2.386405523493990...|

```

	tertiary	N02 2.378265816089016E10
	secondary	N01 2.353814477291994...
	household	N04 1.898439940807995E10
	tertiary	N03 1.759486886659994...
	tertiary	N04 1.470583001795001...
	household	N05 1.411890948306995E10
	tertiary	N05 1.068887617802003E10
	cabin	N01 3.085755719926999E9
	primary	N03 2.9519108608209987E9
	cabin	N02 2.503069983573E9
	primary	N02 2.464051049340006E9
	primary	N04 1.995885701865011E9

+-----+
only showing top 20 rows

Production Statistics by Group:

25/11/27 09:20:06 WARN TaskSetManager: Stage 32 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

consumptionGroup	total_kwh	avg_kwh
max_kwh record_count		
secondary 2.372117350413903...	1353021.5322917541	2734083.2
175320		
household 1.522672474538196...	868510.4235330803	4290175.0
175320		
tertiary 1.077519523341402...	614601.5989855139	2524264.0
175320		
cabin 1.002643229311401...	57189.324053810255	238391.33
175320		
primary 9.646897669629007E9	55024.51328786794	129117.58
175320		

25/11/27 09:20:07 WARN TaskSetManager: Stage 40 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

Spark aggregations completed

Summary records: 25

Stats records: 5

Sample summary data:

	consumptionGroup	priceArea	total_production
0	secondary	N02	7.670675e+10
1	secondary	N03	6.213564e+10
2	household	N01	6.069843e+10
3	tertiary	N01	4.097972e+10
4	secondary	N05	3.758356e+10
5	secondary	N04	3.724764e+10
6	household	N02	3.460145e+10
7	household	N03	2.386406e+10
8	tertiary	N02	2.378266e+10
9	secondary	N01	2.353814e+10

1.7 Inserting data into MongoDB

```
[12]: import streamlit as st
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi

uri = st.secrets["database"]["uri"]
# Create a new client and connect to the server
client = MongoClient(uri, server_api=ServerApi('1'))

# Send a ping to confirm a successful connection
try:
    client.admin.command('ping')
    print("Pinged your deployment. You successfully connected to MongoDB!")
except Exception as e:
    print(e)
```

Pinged your deployment. You successfully connected to MongoDB!

1.7.1 Connecting

```
[13]: # Selecting a database and a collection.
DB = client['ELBHUB_Data']
Production_Collection = DB['production_data']
Consumption_Collection = DB['consumption_data']
```

1.7.2 Inserting

```
[14]: def insert_data_into_mongodb(collection, spark_df, db, batch_size=10000):
    # Convert Spark DataFrame to pandas DataFrame
    df = spark_df.toPandas()
    # First, clear the existing collection to avoid duplicates
    collection.delete_many({})
    print("Cleared existing data from collection")

    # Convert datetime columns to ISO string format, set missing as None
```

```

datetime_columns = ['startTime', 'endTime', 'lastUpdatedTime']
for col in datetime_columns:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors='coerce')

# Convert the Pandas DataFrame to a list of dictionaries
data_to_insert = df.to_dict(orient='records')

print(f"\nPreparing to insert {len(data_to_insert)} documents in batches of {batch_size}...")
if data_to_insert:
    print(f"Sample document: {data_to_insert[0]}")

# Insert the data into the MongoDB collection in batches
total_inserted = 0
for i in range(0, len(data_to_insert), batch_size):
    batch = data_to_insert[i:i+batch_size]
    collection.insert_many(batch)
    total_inserted += len(batch)
    print(f"Inserted batch {i//batch_size + 1}: {len(batch)} documents (Total: {total_inserted})")

print(f"\n Successfully inserted {len(data_to_insert)} documents into the MongoDB collection.")
print(f"Database: {db.name}")
print(f"Collection: {collection.name}")

insert_data_into_mongodb(Production_Collection, spark_production_df, DB)
insert_data_into_mongodb(Consumption_Collection, spark_consumption_df, DB)

```

25/11/27 09:20:11 WARN TaskSetManager: Stage 48 contains a task of very large size (6056 KiB). The maximum recommended task size is 1000 KiB.

Cleared existing data from collection

Preparing to insert 872953 documents in batches of 10000...

Sample document: {'endTime': Timestamp('2021-01-01 00:00:00'),
 'lastUpdatedTime': Timestamp('2024-12-20 09:35:40'), 'priceArea': 'N01',
 'productionGroup': 'hydro', 'quantityKwh': 2507716.8, 'startTime':
 Timestamp('2020-12-31 23:00:00'), 'row_id': 1}

Inserted batch 1: 10000 documents (Total: 10000)
 Inserted batch 2: 10000 documents (Total: 20000)
 Inserted batch 3: 10000 documents (Total: 30000)
 Inserted batch 4: 10000 documents (Total: 40000)
 Inserted batch 5: 10000 documents (Total: 50000)
 Inserted batch 6: 10000 documents (Total: 60000)
 Inserted batch 7: 10000 documents (Total: 70000)
 Inserted batch 8: 10000 documents (Total: 80000)

Inserted batch 9: 10000 documents (Total: 90000)
Inserted batch 10: 10000 documents (Total: 100000)
Inserted batch 11: 10000 documents (Total: 110000)
Inserted batch 12: 10000 documents (Total: 120000)
Inserted batch 13: 10000 documents (Total: 130000)
Inserted batch 14: 10000 documents (Total: 140000)
Inserted batch 15: 10000 documents (Total: 150000)
Inserted batch 16: 10000 documents (Total: 160000)
Inserted batch 17: 10000 documents (Total: 170000)
Inserted batch 18: 10000 documents (Total: 180000)
Inserted batch 19: 10000 documents (Total: 190000)
Inserted batch 20: 10000 documents (Total: 200000)
Inserted batch 21: 10000 documents (Total: 210000)
Inserted batch 22: 10000 documents (Total: 220000)
Inserted batch 23: 10000 documents (Total: 230000)
Inserted batch 24: 10000 documents (Total: 240000)
Inserted batch 25: 10000 documents (Total: 250000)
Inserted batch 26: 10000 documents (Total: 260000)
Inserted batch 27: 10000 documents (Total: 270000)
Inserted batch 28: 10000 documents (Total: 280000)
Inserted batch 29: 10000 documents (Total: 290000)
Inserted batch 30: 10000 documents (Total: 300000)
Inserted batch 31: 10000 documents (Total: 310000)
Inserted batch 32: 10000 documents (Total: 320000)
Inserted batch 33: 10000 documents (Total: 330000)
Inserted batch 34: 10000 documents (Total: 340000)
Inserted batch 35: 10000 documents (Total: 350000)
Inserted batch 36: 10000 documents (Total: 360000)
Inserted batch 37: 10000 documents (Total: 370000)
Inserted batch 38: 10000 documents (Total: 380000)
Inserted batch 39: 10000 documents (Total: 390000)
Inserted batch 40: 10000 documents (Total: 400000)
Inserted batch 41: 10000 documents (Total: 410000)
Inserted batch 42: 10000 documents (Total: 420000)
Inserted batch 43: 10000 documents (Total: 430000)
Inserted batch 44: 10000 documents (Total: 440000)
Inserted batch 45: 10000 documents (Total: 450000)
Inserted batch 46: 10000 documents (Total: 460000)
Inserted batch 47: 10000 documents (Total: 470000)
Inserted batch 48: 10000 documents (Total: 480000)
Inserted batch 49: 10000 documents (Total: 490000)
Inserted batch 50: 10000 documents (Total: 500000)
Inserted batch 51: 10000 documents (Total: 510000)
Inserted batch 52: 10000 documents (Total: 520000)
Inserted batch 53: 10000 documents (Total: 530000)
Inserted batch 54: 10000 documents (Total: 540000)
Inserted batch 55: 10000 documents (Total: 550000)
Inserted batch 56: 10000 documents (Total: 560000)

Inserted batch 57: 10000 documents (Total: 570000)
Inserted batch 58: 10000 documents (Total: 580000)
Inserted batch 59: 10000 documents (Total: 590000)
Inserted batch 60: 10000 documents (Total: 600000)
Inserted batch 61: 10000 documents (Total: 610000)
Inserted batch 62: 10000 documents (Total: 620000)
Inserted batch 63: 10000 documents (Total: 630000)
Inserted batch 64: 10000 documents (Total: 640000)
Inserted batch 65: 10000 documents (Total: 650000)
Inserted batch 66: 10000 documents (Total: 660000)
Inserted batch 67: 10000 documents (Total: 670000)
Inserted batch 68: 10000 documents (Total: 680000)
Inserted batch 69: 10000 documents (Total: 690000)
Inserted batch 70: 10000 documents (Total: 700000)
Inserted batch 71: 10000 documents (Total: 710000)
Inserted batch 72: 10000 documents (Total: 720000)
Inserted batch 73: 10000 documents (Total: 730000)
Inserted batch 74: 10000 documents (Total: 740000)
Inserted batch 75: 10000 documents (Total: 750000)
Inserted batch 76: 10000 documents (Total: 760000)
Inserted batch 77: 10000 documents (Total: 770000)
Inserted batch 78: 10000 documents (Total: 780000)
Inserted batch 79: 10000 documents (Total: 790000)
Inserted batch 80: 10000 documents (Total: 800000)
Inserted batch 81: 10000 documents (Total: 810000)
Inserted batch 82: 10000 documents (Total: 820000)
Inserted batch 83: 10000 documents (Total: 830000)
Inserted batch 84: 10000 documents (Total: 840000)
Inserted batch 85: 10000 documents (Total: 850000)
Inserted batch 86: 10000 documents (Total: 860000)
Inserted batch 87: 10000 documents (Total: 870000)
Inserted batch 88: 2953 documents (Total: 872953)

Successfully inserted 872953 documents into the MongoDB collection.

Database: ELBHUB_Data

Collection: production_data

25/11/27 09:28:25 WARN TaskSetManager: Stage 49 contains a task of very large size (6819 KiB). The maximum recommended task size is 1000 KiB.

Cleared existing data from collection

Preparing to insert 876600 documents in batches of 10000...

Sample document: {'consumptionGroup': 'cabin', 'endTime': Timestamp('2021-01-01 00:00:00'), 'lastUpdatedTime': Timestamp('2024-12-20 09:35:40'), 'meteringPointCount': 100607, 'priceArea': 'N01', 'quantityKwh': 177071.56, 'startTime': Timestamp('2020-12-31 23:00:00'), 'row_id': 1}

Inserted batch 1: 10000 documents (Total: 10000)

Inserted batch 2: 10000 documents (Total: 20000)

Inserted batch 3: 10000 documents (Total: 30000)
Inserted batch 4: 10000 documents (Total: 40000)
Inserted batch 5: 10000 documents (Total: 50000)
Inserted batch 6: 10000 documents (Total: 60000)
Inserted batch 7: 10000 documents (Total: 70000)
Inserted batch 8: 10000 documents (Total: 80000)
Inserted batch 9: 10000 documents (Total: 90000)
Inserted batch 10: 10000 documents (Total: 100000)
Inserted batch 11: 10000 documents (Total: 110000)
Inserted batch 12: 10000 documents (Total: 120000)
Inserted batch 13: 10000 documents (Total: 130000)
Inserted batch 14: 10000 documents (Total: 140000)
Inserted batch 15: 10000 documents (Total: 150000)
Inserted batch 16: 10000 documents (Total: 160000)
Inserted batch 17: 10000 documents (Total: 170000)
Inserted batch 18: 10000 documents (Total: 180000)
Inserted batch 19: 10000 documents (Total: 190000)
Inserted batch 20: 10000 documents (Total: 200000)
Inserted batch 21: 10000 documents (Total: 210000)
Inserted batch 22: 10000 documents (Total: 220000)
Inserted batch 23: 10000 documents (Total: 230000)
Inserted batch 24: 10000 documents (Total: 240000)
Inserted batch 25: 10000 documents (Total: 250000)
Inserted batch 26: 10000 documents (Total: 260000)
Inserted batch 27: 10000 documents (Total: 270000)
Inserted batch 28: 10000 documents (Total: 280000)
Inserted batch 29: 10000 documents (Total: 290000)
Inserted batch 30: 10000 documents (Total: 300000)
Inserted batch 31: 10000 documents (Total: 310000)
Inserted batch 32: 10000 documents (Total: 320000)
Inserted batch 33: 10000 documents (Total: 330000)
Inserted batch 34: 10000 documents (Total: 340000)
Inserted batch 35: 10000 documents (Total: 350000)
Inserted batch 36: 10000 documents (Total: 360000)
Inserted batch 37: 10000 documents (Total: 370000)
Inserted batch 38: 10000 documents (Total: 380000)
Inserted batch 39: 10000 documents (Total: 390000)
Inserted batch 40: 10000 documents (Total: 400000)
Inserted batch 41: 10000 documents (Total: 410000)
Inserted batch 42: 10000 documents (Total: 420000)
Inserted batch 43: 10000 documents (Total: 430000)
Inserted batch 44: 10000 documents (Total: 440000)
Inserted batch 45: 10000 documents (Total: 450000)
Inserted batch 46: 10000 documents (Total: 460000)
Inserted batch 47: 10000 documents (Total: 470000)
Inserted batch 48: 10000 documents (Total: 480000)
Inserted batch 49: 10000 documents (Total: 490000)
Inserted batch 50: 10000 documents (Total: 500000)

Inserted batch 51: 10000 documents (Total: 510000)
Inserted batch 52: 10000 documents (Total: 520000)
Inserted batch 53: 10000 documents (Total: 530000)
Inserted batch 54: 10000 documents (Total: 540000)
Inserted batch 55: 10000 documents (Total: 550000)
Inserted batch 56: 10000 documents (Total: 560000)
Inserted batch 57: 10000 documents (Total: 570000)
Inserted batch 58: 10000 documents (Total: 580000)
Inserted batch 59: 10000 documents (Total: 590000)
Inserted batch 60: 10000 documents (Total: 600000)
Inserted batch 61: 10000 documents (Total: 610000)
Inserted batch 62: 10000 documents (Total: 620000)
Inserted batch 63: 10000 documents (Total: 630000)
Inserted batch 64: 10000 documents (Total: 640000)
Inserted batch 65: 10000 documents (Total: 650000)
Inserted batch 66: 10000 documents (Total: 660000)
Inserted batch 67: 10000 documents (Total: 670000)
Inserted batch 68: 10000 documents (Total: 680000)
Inserted batch 69: 10000 documents (Total: 690000)
Inserted batch 70: 10000 documents (Total: 700000)
Inserted batch 71: 10000 documents (Total: 710000)
Inserted batch 72: 10000 documents (Total: 720000)
Inserted batch 73: 10000 documents (Total: 730000)
Inserted batch 74: 10000 documents (Total: 740000)
Inserted batch 75: 10000 documents (Total: 750000)
Inserted batch 76: 10000 documents (Total: 760000)
Inserted batch 77: 10000 documents (Total: 770000)
Inserted batch 78: 10000 documents (Total: 780000)
Inserted batch 79: 10000 documents (Total: 790000)
Inserted batch 80: 10000 documents (Total: 800000)
Inserted batch 81: 10000 documents (Total: 810000)
Inserted batch 82: 10000 documents (Total: 820000)
Inserted batch 83: 10000 documents (Total: 830000)
Inserted batch 84: 10000 documents (Total: 840000)
Inserted batch 85: 10000 documents (Total: 850000)
Inserted batch 86: 10000 documents (Total: 860000)
Inserted batch 87: 10000 documents (Total: 870000)
Inserted batch 88: 6600 documents (Total: 876600)

Successfully inserted 876600 documents into the MongoDB collection.
Database: ELBHUB_Data
Collection: consumption_data

1.8 Word Log

Starting of, I decived to have the API fetching just adopt the same method as for CA2 but then download all data for the period 2021-2024. So its now all fetched again -> Stored as a JSON file -> Stored in Cassandra -> Retreived from Cassandra with Spark, then inserted to MongoDB.

Found this more senseful than making code to just append the last three years. Also now have proper functions to use for extractions of any year.

I also had some major issues with MongoDB, and tried to debug my pipeline for 2 days straight, but after attending the exercise lesson thursday 27.nov i learned that MongoDB had a Data transer limit of 10GB per 7 days. It then became appearant that I might have exceeded this limit in my many reloads of this application for the final handin. So after upgrading my mongoDB subscription (just to be sure i could reload as much as i want for the last few days) my speeds were back to normal.

For the Bonus task i chose to do some spinners and error messages, also some caching and also the monthly snow drift plot-> A good mix.

This course project have led on several challenges. The mongoDB thing being the most frustrating one, but also with understanding the data and accessing it properly throughout the app and the many operations in order to plot.

This course, being so encouraging towards the use of AI, has also allowed be to experiment with the phenomenon of “vibe-coding” and its many many downsizes... Rest to asure, this project showed me a lot of how amazing AI can be when used properly, but also how useless it sometimes is when used improperly.

For this last CA i also had to ask my classmates more about their approach in order to get my app working. In this process ive shared my code, and also “borrowed” code from my classmates to use in this assignment.

Lastly for the Meteorology i found correlation to be the highest with temperatures and solar energy production, which makes a lot of sence.